

1. Introduction to Data Structures and Algorithms

Topics

1.1 Data Structures concepts: Definition, classification, and need for data structures. Types of data structures: primitive, non-primitive, linear, and nonlinear, Abstract Data Types (ADT)

1.2 Concept of algorithms: properties, design techniques, and performance analysis. Asymptotic notation: Big O, Omega, Theta with examples

Self-Learning: Comparative growth analysis of functions

Data Structures concepts

Definition:

- A data structure is a specialized format for organizing and storing data in a computer so that it can be accessed and modified efficiently.
- Data Structure can be defined as the group of data elements which provides an efficient way of storing and organizing data in the computer so that it can be used efficiently.
- A Data Structure is a systematic way of organizing, storing, and managing data in a computer so that it can be accessed, modified, and processed efficiently.
- A data structure is a way to store data.

Need for data structures

Efficient Data Access : Faster searching, insertion, deletion, and updating.

Better Memory Management : Optimal use of memory.

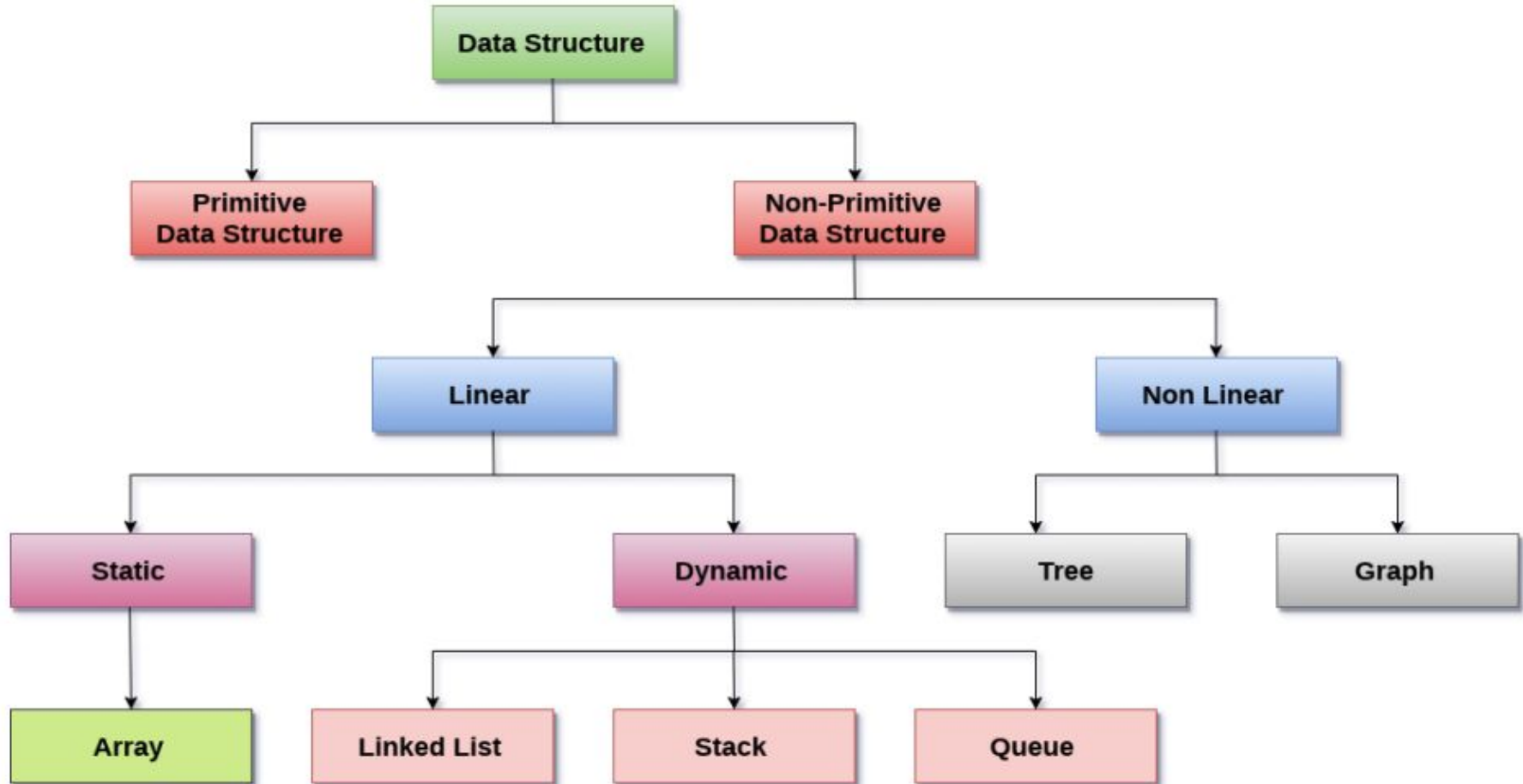
Efficient Algorithms : Good data structures improve algorithm performance.

Handling Large Data – Helps manage large volumes of data easily.

Data Organization – Maintains data in a structured manner.

Problem Solving – Real-world problems can be solved efficiently.

Classification of Data Structures



Primitive Data Structure -

Primitive Data Structures directly operate according to the machine instructions. These are the primitive data types. Data types like int, char, float, double, and pointer are primitive data structures that can hold a single value.

Non - Primitive Data Structure -

Non-primitive data structures are complex data structures that are derived from primitive data structures. Non - Primitive data types are further divided into two categories.

Linear Data Structure

Non - Linear Data Structure

Linear data structure: Data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure.

Examples: array, stack, queue, linked list, etc.

Static data structure: Static data structure has a fixed memory size. It is easier to access the elements in a static data structure.

Example: array data structure.

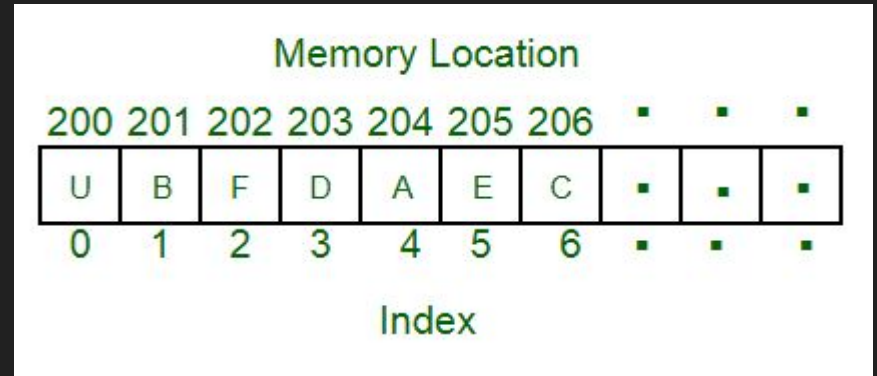
Dynamic data structure: In the dynamic data structure, the size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (space) complexity of the code.

Examples: stack and queue data structures.

Non-linear data structure: Data structures where data elements are not placed sequentially or linearly are called non-linear data structures. In a non-linear data structure, we can't traverse all the elements in a single run.

Examples: tree and graph data structures.

Arrays Data Structure



An array is a linear data structure and it is a collection of element of same data type stored at contiguous memory locations.

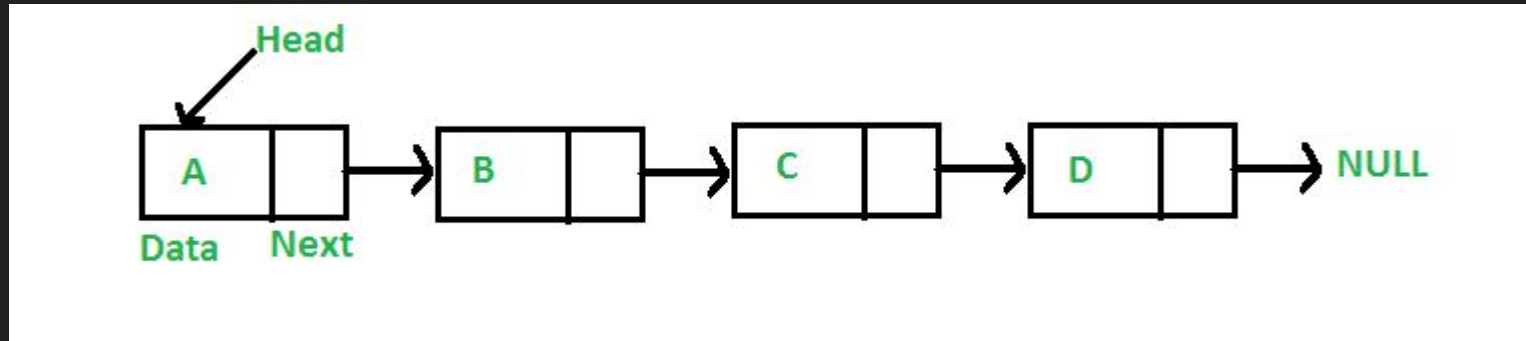
It offers mainly the following advantages.

Random Access: i -th elements can be accessed in $O(1)$ Time as we have the base address and every element is of same size.

Cache Friendliness: Since elements are stored at contiguous locations, we get the advantage of locality of reference.

Linked list Data Structure

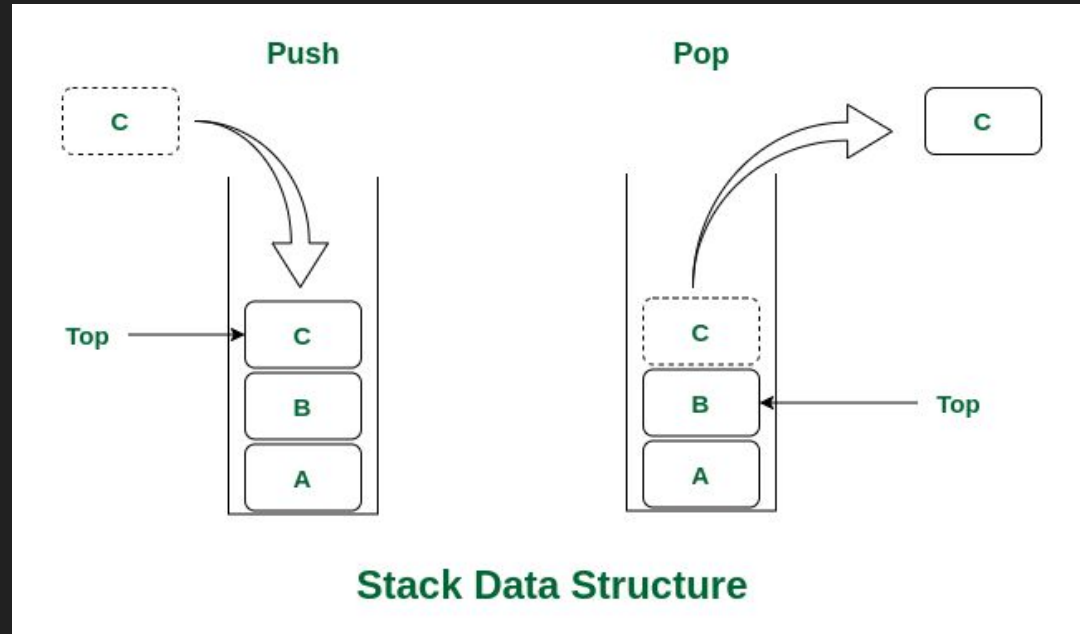
A linked list is a linear data structure in which elements are not stored at contiguous memory locations. The elements in a linked list are linked using pointers as shown in the below image.



Stack Data Structure

Stack is a linear data structure that follows LIFO (Last in first out) principle i.e., entering and retrieving data is possible from only one end.

The entering and retrieving of data is also called push and pop operation in a stack.

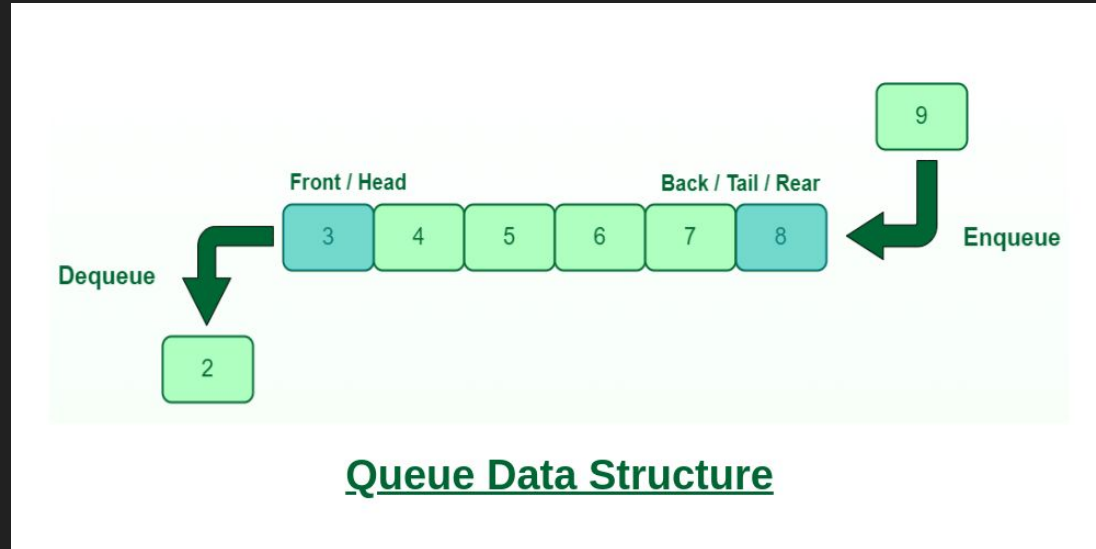


Queue Data Structure

Queue is a linear data structure that follows First In First Out (FIFO) principle i.e. the data item stored first will be accessed first.

In this, entering is done from one end and retrieving data is done from other end.

An example of a queue is any queue of consumers for a resource where the consumer that came first is served first.



Tree Data Structure

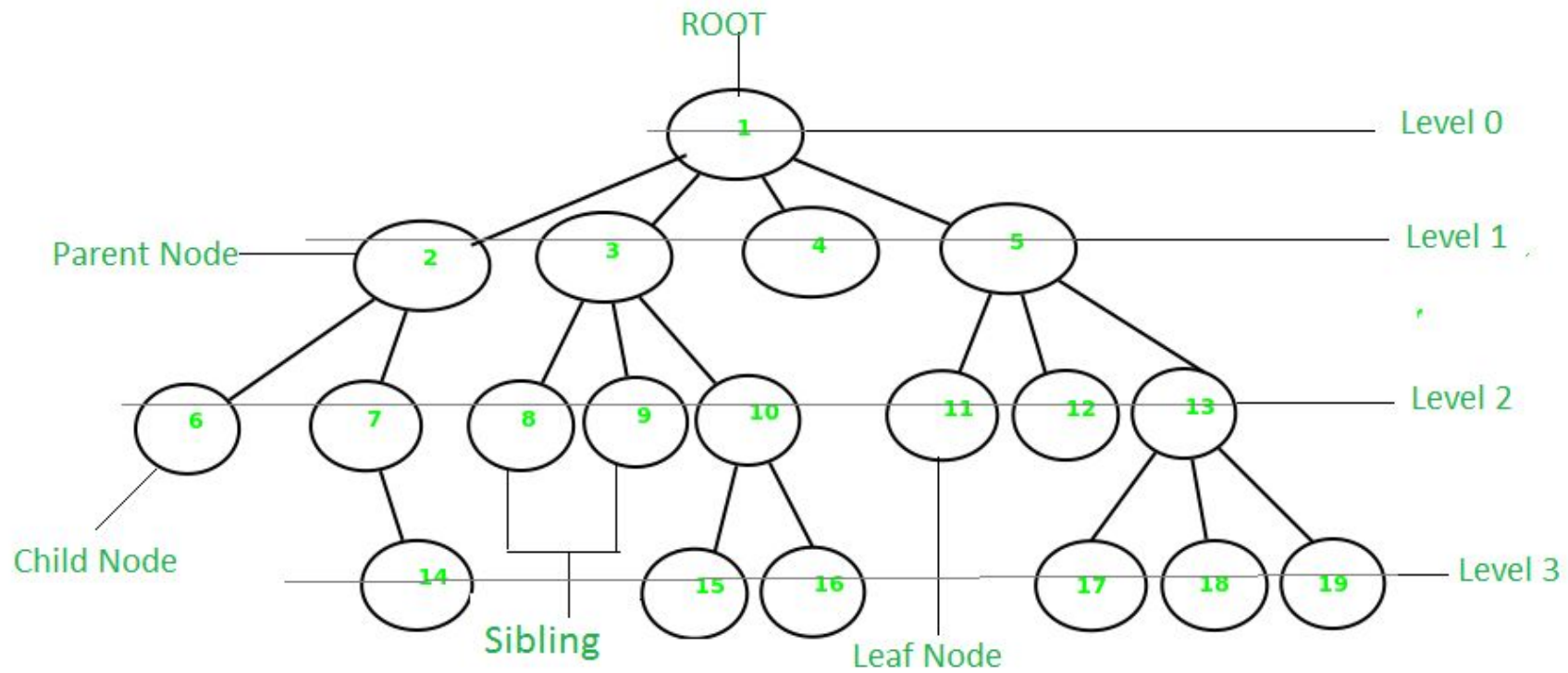
A tree is a non-linear and hierarchical data structure where the elements are arranged in a tree-like structure.

In a tree, the topmost node is called the root node.

Each node contains some data, and data can be of any type.

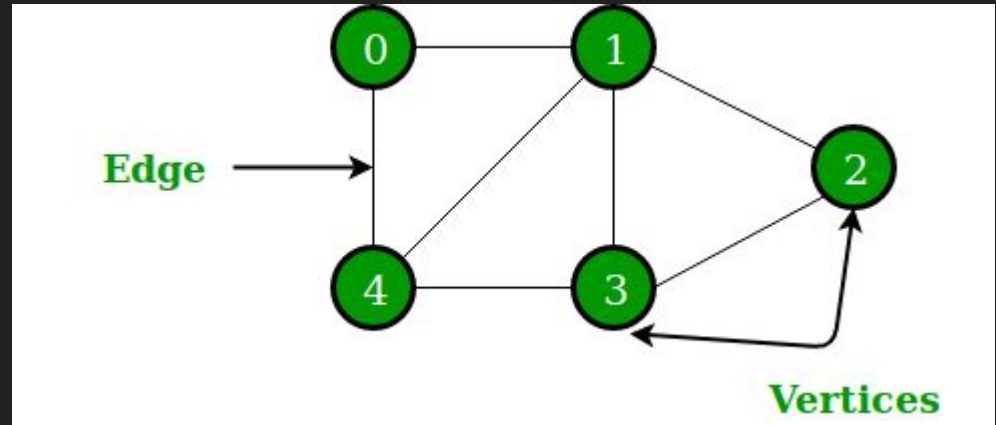
It consists of a central node, structural nodes, and sub-nodes which are connected via edges.

Different tree data structures allow quicker and easier access to the data as it is a non-linear data structure.



Graph Data Structure

A graph is a non-linear data structure that consists of vertices (or nodes) and edges. It consists of a finite set of vertices and set of edges that connect a pair of nodes. The graph is used to solve the most challenging and complex programming problems. It has different terminologies which are Path, Degree, Adjacent vertices, Connected components, etc.



Data Structure Operations

Insertion : It provides the means for adding new details or new node into the existing data structure.

Deletion : It provides the means for removing a node from the data structure.

Traversing : It provides the means for accessing each node exactly once so that the nodes of a data structure can be processed. It is also called the visiting to data structure.

Searching : It provides the means for finding the location of node for a given key value or finding the locations of all records, which satisfy one or more conditions.

Sorting : It provides the means for arranging the data in a particular order in given data structure.

Merging : It provides the means for joining the two data structures.

Advantages of Data Structure

- Efficient Data Processing : Proper data structures reduce time complexity and improve performance.
- Better Memory Management : Optimizes memory usage by storing data in an organized manner.
- Faster Searching and Sorting : Data structures like trees and hash tables provide quick search operations.
- Improved Algorithm Efficiency : Choosing the right data structure improves overall algorithm speed.
- Reusability and Scalability : Data structures support reusable code and can handle growing data sizes.
- Data Organization : Maintains structured and systematic data storage.
- Simplifies Problem Solving : Complex problems become easier to solve using appropriate data structures.
- Improved Data Security : Structured access controls help protect sensitive data.

Abstract Data Types (ADT)

An ADT defines what operations are available and their behavior (e.g., a stack follows LIFO behavior).

A Data Structure defines how data is organized in memory and how those operations are executed (e.g., using an array or a linked list to implement a stack).

Definition:

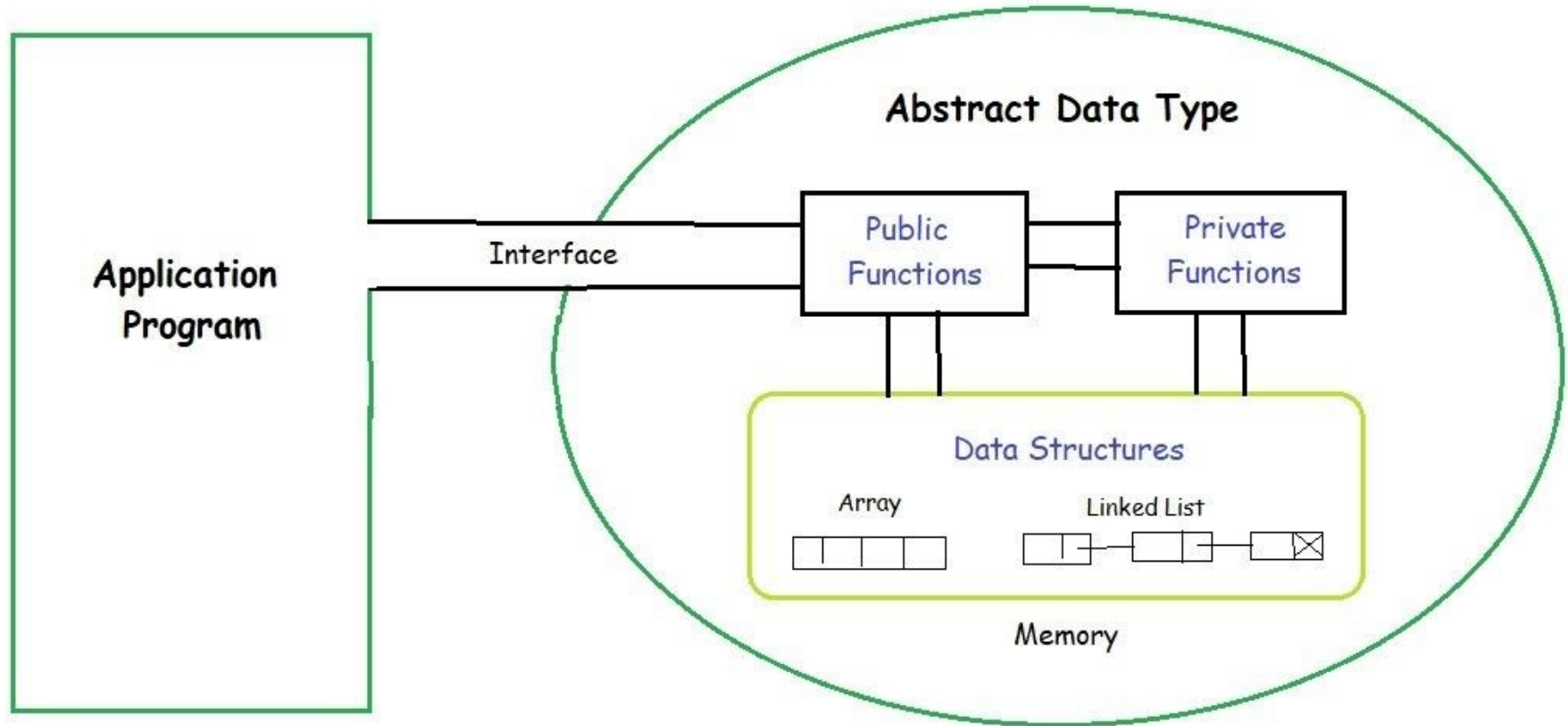
An Abstract Data Type (ADT) is a conceptual model that defines a set of operations and behaviors for a data structure, without specifying how these operations are implemented or how data is organized in memory.

The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented.

It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations.

It is called "abstract" because it provides an implementation-independent view.

Abstract Data Type Model



There are two types of models in the ADT model:

1. public function
2. private function

The ADT model also contains the data structures being used in a program. Here, at first, encapsulation is performed, i.e., all the data is wrapped in a single unit, i.e., ADT. Afterwards, abstraction is performed i.e. showing the operations that can be performed on the data structure and also showcasing the data structures being used in a program.

Key Characteristics of ADT

Abstraction – Implementation details are hidden.

Well-defined operations – Each operation has a clear purpose.

Implementation independent – Can be implemented using different data structures.

Encapsulation – Data and operations are bundled together.

Components of an ADT

An ADT is typically defined by:

Data: The type or structure of the data being stored (e.g., integers in a stack, key-value pairs in a map).

Operations: The functions or methods that can be applied to the data, such as insert, delete, search, push, pop, enqueue, dequeue, etc..

Behavior (Rules): The logical constraints or axioms that govern how the operations behave (e.g., a stack follows the Last-In-First-Out (LIFO) rule).

Abstract Data Types : List ADT:

1. `get()`: Return an element from the list at any given position.
2. `insert()`: Insert an element at any position in the list.
3. `remove()`: Remove the first occurrence of any element from a non-empty list.
4. `removeAt()`: Remove the element at a specified location from a non-empty list.
5. `replace()`: Replace an element at any position with another element.
6. `size()`: Return the number of elements in the list.
7. `isEmpty()`: Return true if the list is empty; otherwise, return false.
8. `isFull()`: Return true if the list is full, otherwise, return false. Only applicable in fixed-size implementations (e.g., array-based lists).

Stack ADT

`push()`: Insert an element at one end of the stack called the top.

`pop()`: Remove and return the element at the top of the stack, if it is not empty.

`peek()`: Return the element at the top of the stack without removing it, if the stack is not empty.

`size()`: Return the number of elements in the stack.

`isEmpty()`: Return true if the stack is empty; otherwise, return false.

`isFull()`: Return true if the stack is full; otherwise, return false. Only relevant for fixed-capacity stacks (e.g., array-based).

Queue ADT

`enqueue()`: Insert an element at the end of the queue.

`dequeue()`: Remove and return the first element of the queue, if the queue is not empty.

`peek()`: Return the element of the queue without removing it, if the queue is not empty.

`size()`: Return the number of elements in the queue.

`isEmpty()`: Return true if the queue is empty; otherwise, return false.

Advantages of ADT

Encapsulation: ADTs provide a way to encapsulate data and operations into a single unit, making it easier to manage and modify the data structure.

Abstraction: ADTs allow users to work with data structures without having to know the implementation details, which can simplify programming and reduce errors.

Data Structure Independence: ADTs can be implemented using different data structures, which can make it easier to adapt to changing needs and requirements.

Information Hiding: ADTs can protect the integrity of data by controlling access and preventing unauthorized modifications.

Modularity: ADTs can be combined with other ADTs to form more complex data structures, which can increase flexibility and modularity in programming.

Disadvantages of ADT

Overhead: Implementing ADTs can add overhead in terms of memory and processing, which can affect performance.

Complexity: ADTs can be complex to implement, especially for large and complex data structures.

Learning Curve: Using ADTs requires knowledge of their implementation and usage, which can take time and effort to learn.

Limited Flexibility: Some ADTs may be limited in their functionality or may not be suitable for all types of data structures.

Cost: Implementing ADTs may require additional resources and investment, which can increase the cost of development.

Concept of algorithms

Algorithm is a set of finite, well-defined steps or instructions designed to solve a problem or perform a computation. It can also be defined as a procedure for solving a mathematical or computational problem in a finite number of steps, often involving repetitive or recursive operations.

In general terms, an algorithm provides a blueprint to write a program to solve a particular problem. It is considered to be an effective procedure for solving a problem in finite number of steps.

Need for Algorithms:

- Solve complex problems efficiently and effectively.
- Automate processes, making them reliable, faster, and easier.
- Enable computers to perform tasks difficult or impossible for humans.
- Widely used in mathematics, computer science, engineering, finance, and data analysis.

Use of the Algorithms

Algorithms are fundamental in solving problems efficiently across various fields:

- **Computer Science:** Basis of programming, from simple sorting and searching to AI and machine learning.
- **Mathematics:** Solve problems like linear equations, shortest paths, and optimization.
- **Operations Research:** Optimize logistics, transportation, and resource allocation.
- **Artificial Intelligence:** Power intelligent systems for tasks like image recognition, NLP, and decision-making.
- **Data Science:** Analyze and extract insights from large datasets in marketing, healthcare, finance, etc.

Properties of algorithm

Clear and Unambiguous: Each step should be precise and lead to only one interpretation.

Well-Defined Inputs: An algorithm may take zero or more inputs, which must be clearly specified.

Well-Defined Outputs: It must produce at least one output, clearly defined.

Finiteness: The algorithm must terminate after a finite number of steps.

Infinite loops or recursion without base conditions violate this property.

Effectiveness: Steps should be simple, feasible, and executable with available resources.

Deterministic: For the same input, the algorithm should always produce the same output.

Language Independent: Algorithms are plain instructions and can be implemented in any programming language, producing the same result.

How to express an Algorithm?

Natural Language: Written in plain English. Easy to describe, but can be unclear for complex problems.

Flowchart: Graphical representation of steps. Easier to visualize than natural language.

Pseudocode: Text-based, code-like instructions without language syntax.

Best way to express an algorithm.

Understandable even to beginners with basic knowledge.

DIFFERENT APPROACHES TO DESIGNING AN ALGORITHM

A complex algorithm is often divided into smaller units called modules.

The process of dividing an algorithm into modules is called modularization.

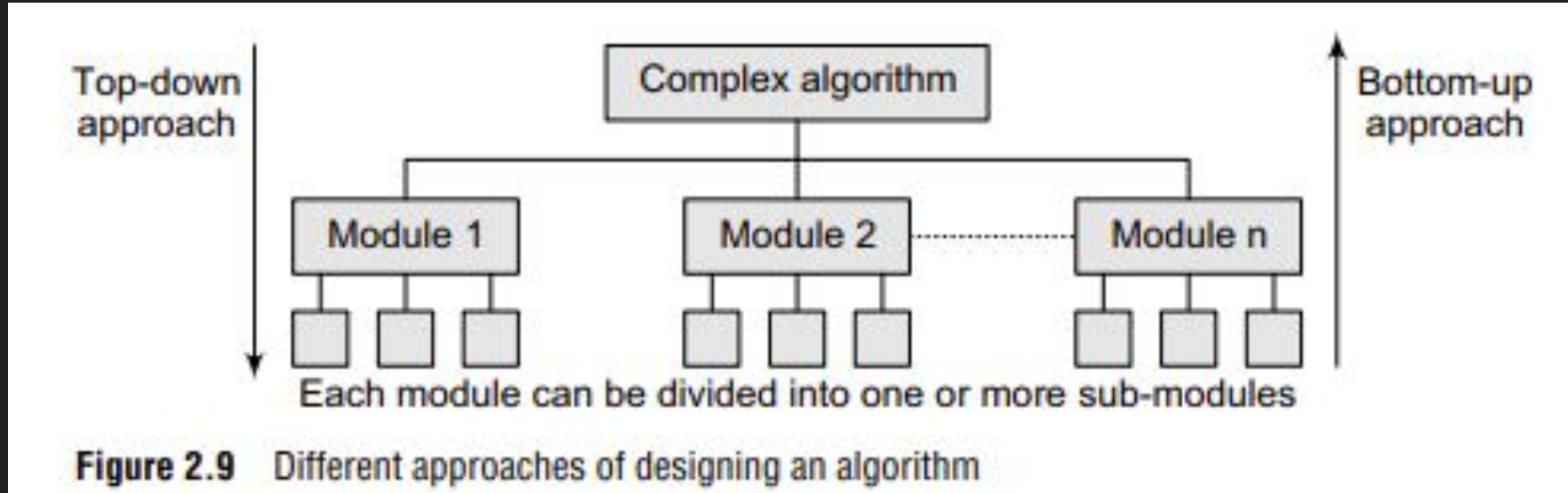
The key advantages of modularization are as follows:

- It makes the complex algorithm simpler to design and implement.
- Each module can be designed independently.
- While designing one module, the details of other modules can be ignored, thereby enhancing clarity in design which in turn simplifies implementation, debugging, testing, documenting, and maintenance of the overall algorithm.

Different approaches of designing an algorithm

There are two main approaches to design an algorithm

- top-down approach
- bottom-up approach



Top-down approach

A top-down design approach starts by dividing the complex algorithm into one or more modules.

These modules can further be decomposed into one or more sub-modules and this process of decomposition is iterated until the desired level of module complexity is achieved.

Top-down design method is a form of stepwise refinement where we begin with the topmost module and incrementally add modules that it calls.

The Top-Down approach works by:

- Breaking the problem into major components,
- Refining each component into smaller subcomponents
- We Continue this process until every part is simple enough to implement.

Bottom-up approach

- A bottom-up approach is just the reverse of top-down approach.
- In the bottom-up design, we start with designing the most basic or concrete modules and then proceed towards designing higher level modules.
- The higher level modules are implemented by using the operations performed by lower level modules.
- Thus, in this approach sub-modules are grouped together to form a higher level module.
- All the higher level modules are clubbed together to form even higher level modules.
- This process is repeated until the design of the complete algorithm is obtained.

Bottom-Up approach works by:

Identifying and specifying the smaller components (or objects).

Linking these smaller parts together to form larger components.

Continuously integrating them to complete the system.

S. No.	TOP DOWN APPROACH	BOTTOM UP APPROACH
1.	focus on breaking the problem into smaller, more manageable parts	Solves smaller problems and integrates them into a complete system
2.	Mainly used in Structured programming languages like COBOL, Fortran, C, etc.	Mainly used in Object Oriented Programming languages like C++, C#, Python.
3.	It is used for debugging and module documentation	It is basically used in testing.
4.	Decomposition of the system occurs, breaking it into smaller components.	Composition of the system happens by combining low-level components into a higher-level structure.
5	Each part is programmed separately therefore contains redundancy.	Redundancy is minimized by using data encapsulation and data hiding.
6	Communication is less among modules.	Modules must communicate to integrate the system.

Algorithm Design Techniques

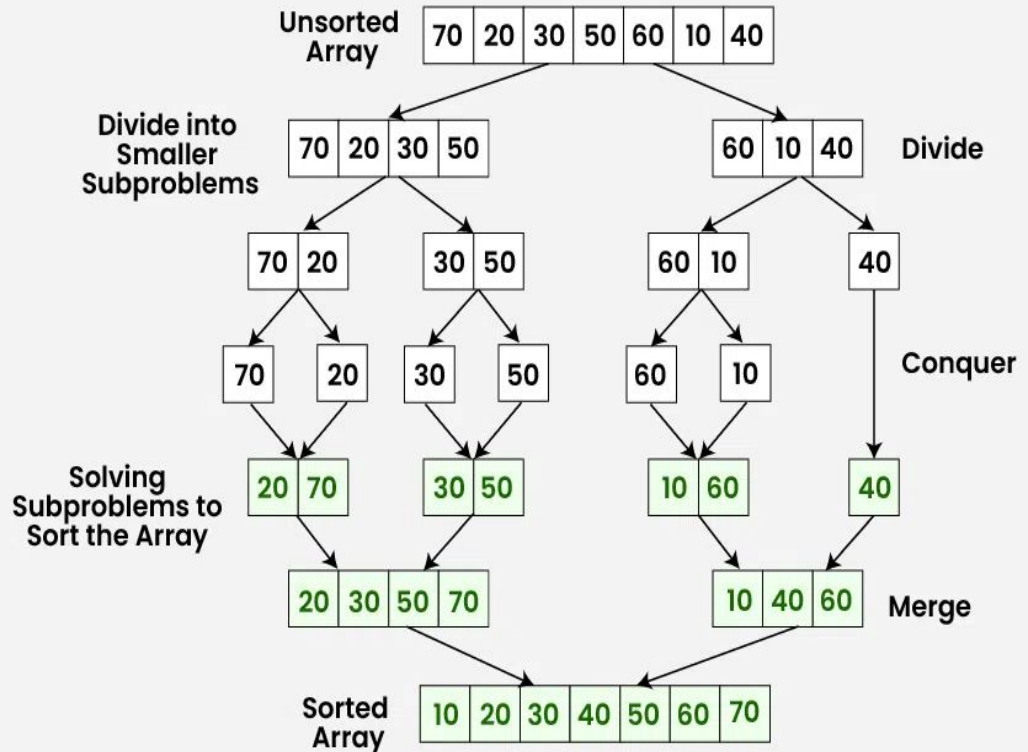
Algorithm design is a critical aspect of problem-solving in computer science. It involves creating systematic methods to solve problems efficiently and effectively. There are several well-known algorithm design techniques that help in developing robust algorithms for a variety of computational problems.

1. Divide and Conquer:

Divide and Conquer is a powerful algorithm design technique that works by **breaking a problem into smaller, more manageable parts**.

Each smaller part is **solved independently**, and then their solutions are **combined to solve** the original problem, as shown in Figure .

This approach is particularly effective for problems that can be divided into similar smaller problems, making it easier to find a solution step by step.



2. Greedy Algorithms

- Greedy is an algorithmic paradigm that builds up a **solution piece by piece**, always **selecting the next piece that offers the most obvious and immediate benefit**.
- Greedy algorithms are used for **optimization problems**.
- Greedy algorithms work by making a **sequence of choices**, each of which is **locally optimal**, with the hope that these choices will lead to a **globally optimal solution**.

Giving Change in a Shop

- A shopkeeper, when returning change, will usually give you the largest coin or note possible first, then smaller ones.
- Example: To return Rs. 37, the shopkeeper gives Rs. 20 + Rs. 10 + Rs. 5 + Rs. 2, instead of many Rs. 1 coins.

3. Dynamic Programming

Dynamic Programming is an algorithmic technique that solves problems by **breaking them into overlapping subproblems, solving each subproblem once, and storing their results to obtain the final solution efficiently.**

The approach of Dynamic programming is similar to divide and conquer. The difference is that whenever we have recursive function calls with the same result, instead of calling them again we try to **store the result** in a data structure in the form of a table and retrieve the results from the table.

Thus, the overall time complexity is reduced. "Dynamic" means we dynamically decide, whether to call a function or retrieve values from the table.

Example:

Few Fibonacci Numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ..

$$F(0)=0$$

$$F(1)=1$$

$$F(2)=F(1)+F(0)=1+0=1$$

$$F(3)=F(2)+F(1)=1+1=2$$

$$F(4)=F(3)+F(2)=2+1=3$$

$$F(5)=F(4)+F(3)=3+2=5$$

$$F(6)=F(5)+F(4)=5+3=8$$

4. Backtracking:

This technique is very useful in solving combination problems that have a **single unique solution**. Where we have to find the correct combination of steps that lead to fulfillment of the task.

Such problems have multiple stages and there are multiple options at each stage. This approach is based on exploring each available option at every stage one-by-one.

While exploring an option if a point is reached that doesn't seem to lead to the solution, the program control backtracks one step, and starts exploring the next option.

In this way, the program explores all possible course of actions and finds the route that leads to the solution.

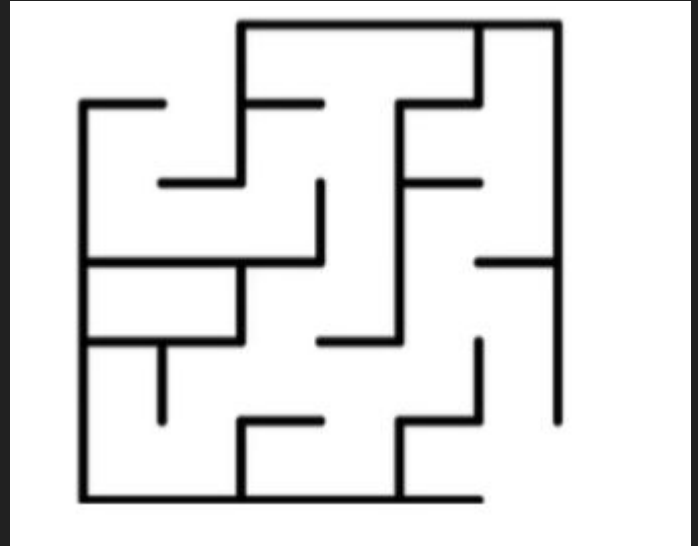
Working Principle:

Choose a possible option

Check whether it is valid

If valid, continue building the solution

If invalid, backtrack and try another option



Analysis of Algorithms

Analysis of Algorithms is a fundamental aspect of computer science that involves evaluating performance of algorithms and programs. Efficiency is measured in terms of time and space.

Performance of an algorithm is a process of making evaluative judgement about algorithms.

Performance of an algorithm means predicting the resources which are required to an algorithm to perform its task.

That means when we have multiple algorithms to solve a problem, we need to **select a suitable algorithm** to solve that problem.

We compare algorithms with each other which are solving the same problem, to select **the best algorithm**.

To compare algorithms, we use a set of parameters or set of elements like Memory required by that algorithm, Execution speed of that algorithm, Easy to understand, Easy to implement, etc.,

Generally, the performance of an algorithm depends on the following elements...

- Whether that algorithm is providing the exact solution for the problem?
- Whether it is easy to understand?
- Whether it is easy to implement?
- How much space (memory) it requires to solve the problem?
- How much time it takes to solve the problem? Etc.,

Performance analysis of an algorithm is the process of calculating space and time required by that algorithm.

Space Complexity

When we design an algorithm to solve a problem, it needs some computer memory to complete its execution. For any algorithm, memory is required for the following purposes...

- To store program instructions.
- To store constant values.
- To store variable values.
- And for few other things like function calls, jumping statements etc,.

Space complexity of an algorithm can be defined as follows...

Total amount of computer memory required by an algorithm to complete its execution is called as space complexity of that algorithm.

Time Complexity

Every algorithm requires some amount of computer time to execute its instruction to perform the task. This computer time required is called time complexity.

The time complexity of an algorithm can be defined as follows...

The time complexity of an algorithm is the **total amount of time required** by an algorithm to complete its execution.

Generally, the running time of an algorithm depends upon the following...

- Whether it is running on Single processor machine or Multi processor machine.
- Whether it is a 32 bit machine or 64 bit machine.
- Read and Write speed of the machine.
- Input data
- The amount of time required by an algorithm to perform Arithmetic operations, logical operations, return value and assignment operations etc.,

Asymptotic Notations

Asymptotic notation of an algorithm is a mathematical representation of its complexity.

Majorly, we use THREE types of Asymptotic Notations and those are as follows...

- Big - Oh (O)
- Big - Omega (Ω)
- Big - Theta (Θ)

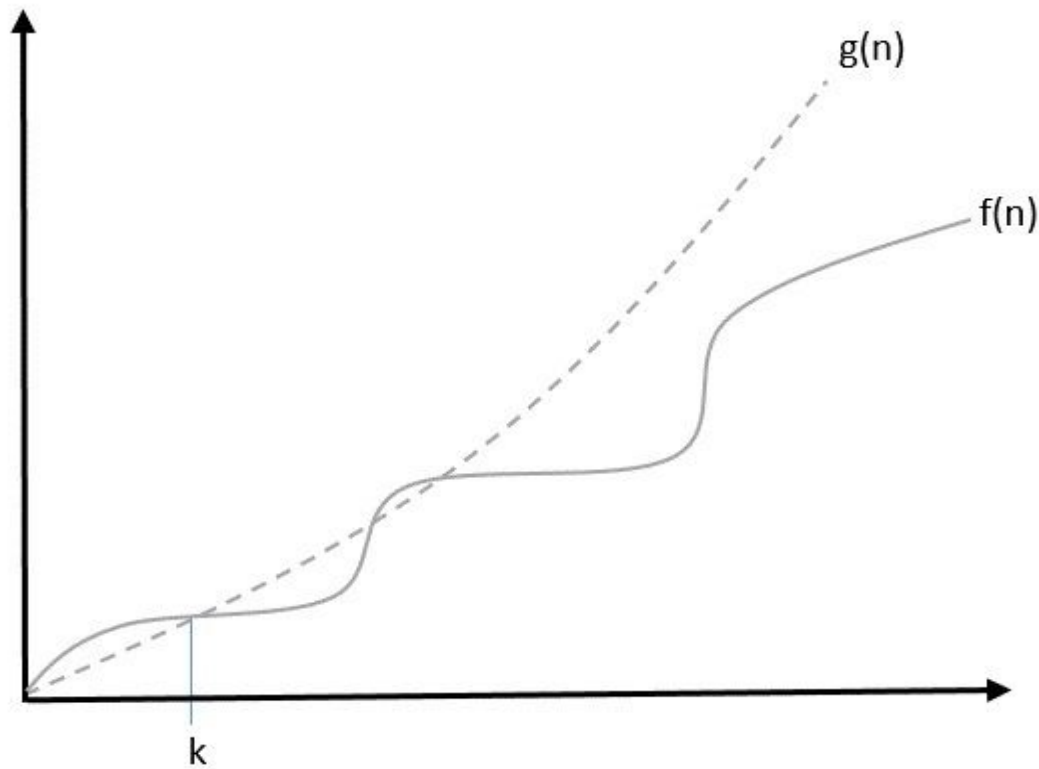
Big - Oh Notation (O):

The notation $O(n)$ is the formal way to express the **upper bound** of an algorithm's running time. It is the most commonly used notation. It measures the **worst case** time complexity or the longest amount of time an algorithm can possibly take to complete.

A function $f(n)$ can be represented is the order of $g(n)$ that is $O(g(n))$, if there exists a value of positive integer n as n_0 and a positive constant c such that –

$$f(n) \leq c \cdot g(n) \quad \text{for } n > n_0 \quad \text{in all case}$$

Hence, function $g(n)$ is an upper bound for function $f(n)$, as $g(n)$ grows faster than $f(n)$.



$O(g(n)) = \{ f(n): \text{there exist positive constants } c \text{ and } n_0$
such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0 \}$

Meaning

- Represents the **upper bound** of an algorithm's time (or space) complexity
- Describes the **worst-case performance**

Definition

An algorithm with time complexity **$f(n) = O(g(n))$** if there exist constants **$c > 0$** and **$n_0 \geq 0$** such that

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

Example

- Linear search $\rightarrow O(n)$ (worst case: element at last position)
- Bubble sort $\rightarrow O(n^2)$

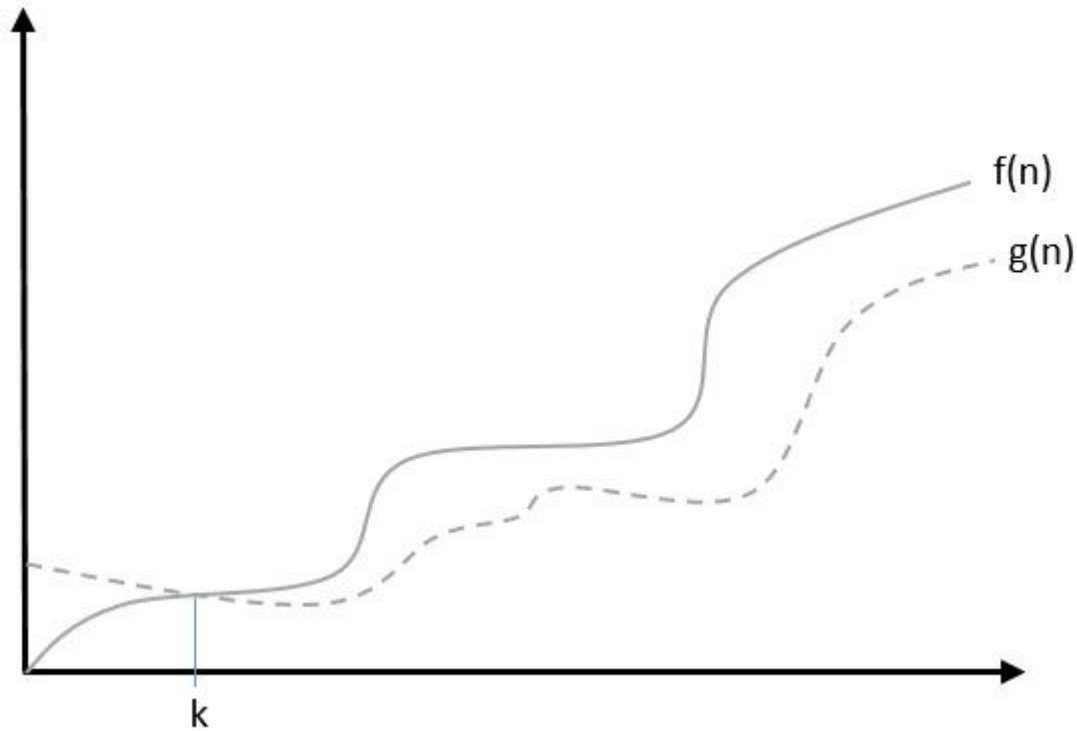
Big Omega, Ω : Asymptotic Lower Bound

The notation $\Omega(n)$ is the formal way to express the **lower bound** of an algorithm's running time. It measures the **best case** time complexity or the best amount of time an algorithm can possibly take to complete.

We say that $f(n)=\Omega(g(n))$

when there exists constant c that $f(n)\geq c.g(n)$

for all sufficiently large value of n . Here n is a positive integer. It means function g is a lower bound for function f ; after a certain value of n , f will never go below g .



$\Omega(g(n)) = \{ f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0 \}$

Meaning

Represents the lower bound of an algorithm's time complexity

Describes the best-case performance

Definition

An algorithm with time complexity $f(n) = \Omega(g(n))$ if

there exist constants $c > 0$ and $n_0 \geq 0$ such that

$f(n) \geq c \cdot g(n)$ for all $n \geq n_0$

Example

Linear search $\rightarrow \Omega(1)$ (best case: element found at first position)

Bubble sort $\rightarrow \Omega(n)$ (already sorted array)

Theta Notation (Θ -notation)

Theta notation encloses the function from **above and below**. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.

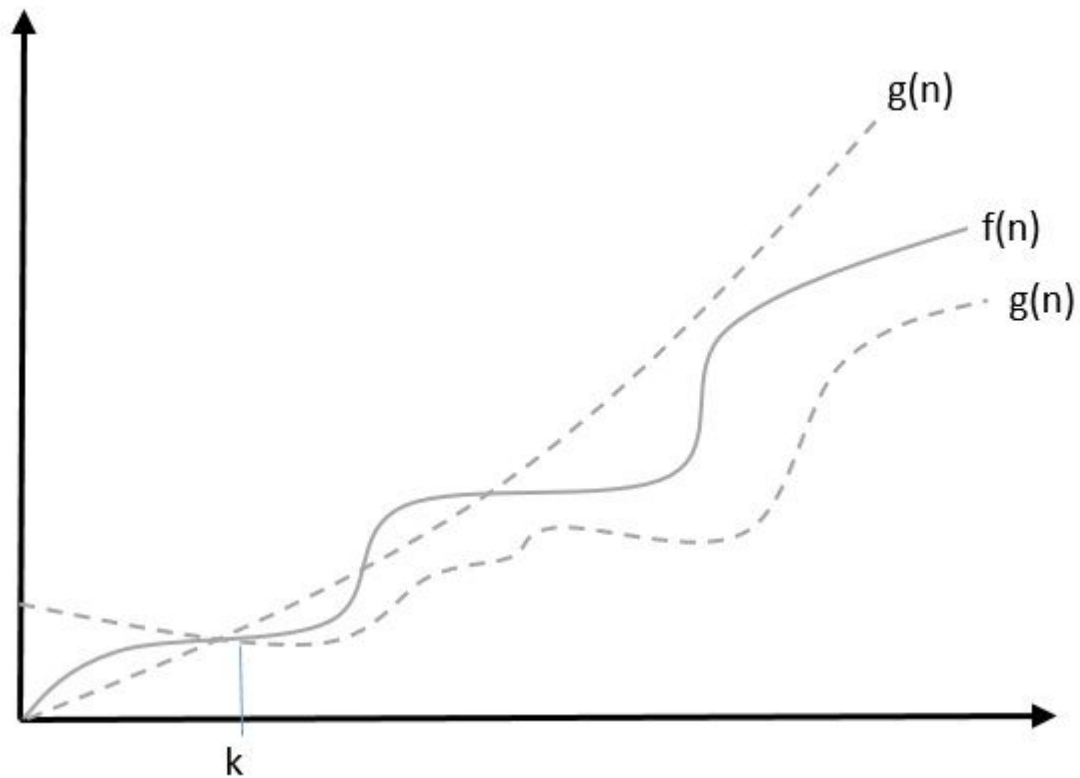
Theta (Average Case) You add the running times for each possible input combination and take the average in the average case.

Theta, θ : Asymptotic Tight Bound

The notation $\theta(n)$ is the formal way to express both the lower bound and the upper bound of an algorithm's running time. Some may confuse the theta notation as the average case time complexity; while big theta notation could be almost accurately used to describe the average case, other notations could be used as well.

We say that $f(n) = \theta(g(n))$ when there exist constants c_1 and c_2 that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all sufficiently large value of n . Here n is a positive integer.

This means function g is a tight bound for function f .



$\Theta(g(n)) = \{ f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0$
 such that $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0 \}$

Meaning

Represents the tight bound

Describes the average / exact growth rate

When upper and lower bounds are the same

Definition

An algorithm with time complexity $f(n) = \Theta(g(n))$ if
there exist constants $c_1, c_2 > 0$ and $n_0 \geq 0$ such that
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$

♦ Example

Linear search $\rightarrow \Theta(n)$

Binary search $\rightarrow \Theta(\log n)$

Aspect	Big-O (O)	Big-Omega (Ω)	Big-Theta (Θ)
Type of bound	Upper bound	Lower bound	Tight bound
Case	Worst case	Best case	Average / exact
Performance view	Maximum time	Minimum time	Exact growth
Usage	Performance guarantee	Minimum cost	Precise analysis

End of Module-1